

Prueba Técnica – Backend Telco Fija Hogar

Java 21 + Spring Boot 3 · PostgreSQL 16 · Docker · JWT

1. Resumen del proyecto

Flujo de ventas Telco Fija Hogar de punta a punta: un AGENTE registra la venta, un BACKOFFICE la aprueba o rechaza (con motivo), y un SUPERVISOR consulta las ventas de su equipo con filtros y un resumen agregado. Incluye además un panel de administración de usuarios (crear, editar, activar/desactivar, resetear contraseña) construido sobre el mismo patrón de capas.

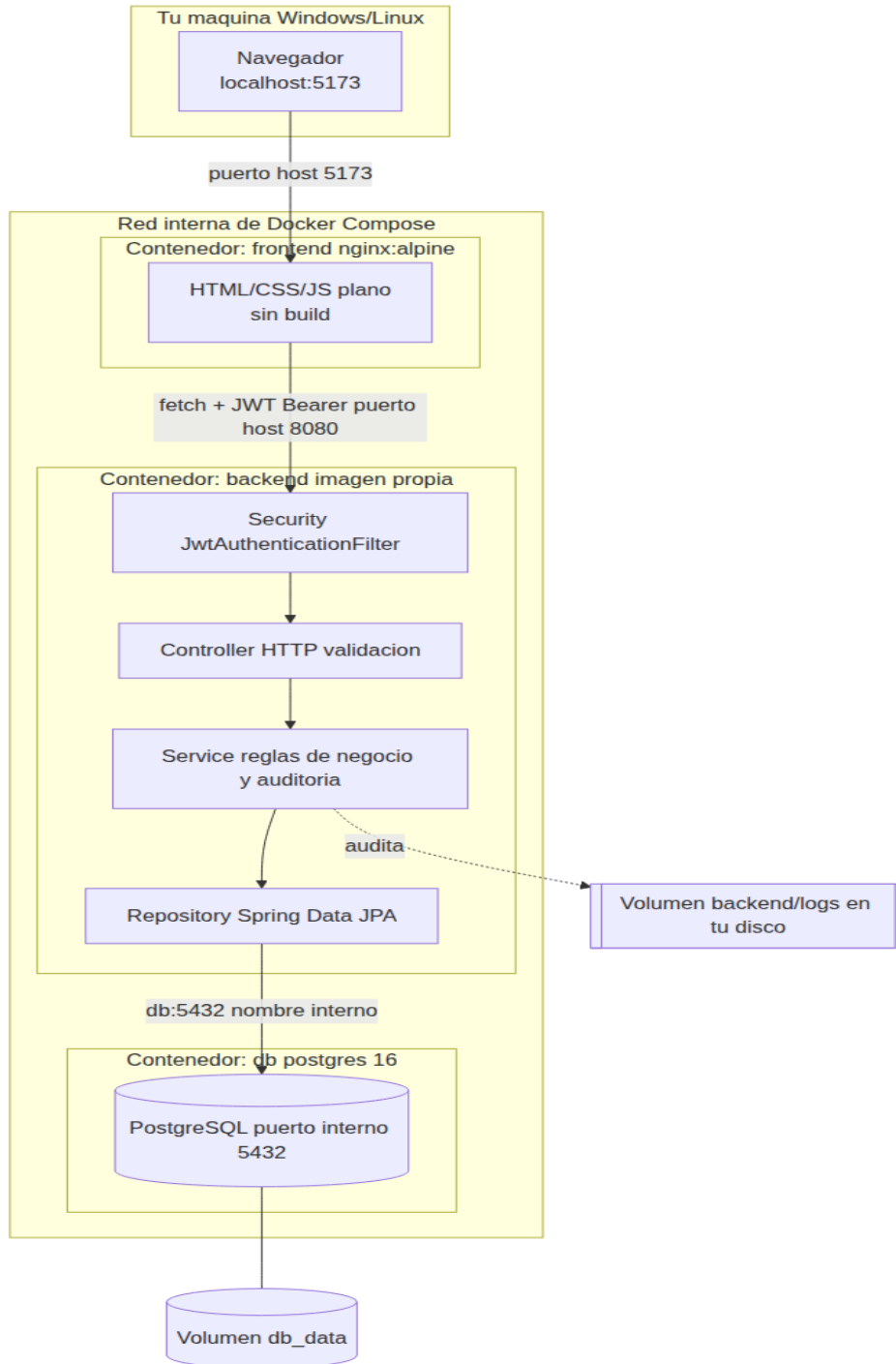
El sistema completo -base de datos, backend y frontend- corre con un solo comando de Docker Compose, sin necesitar Java, Maven ni Node.js instalados en la máquina de quien lo revise.

2. Estructura del repositorio

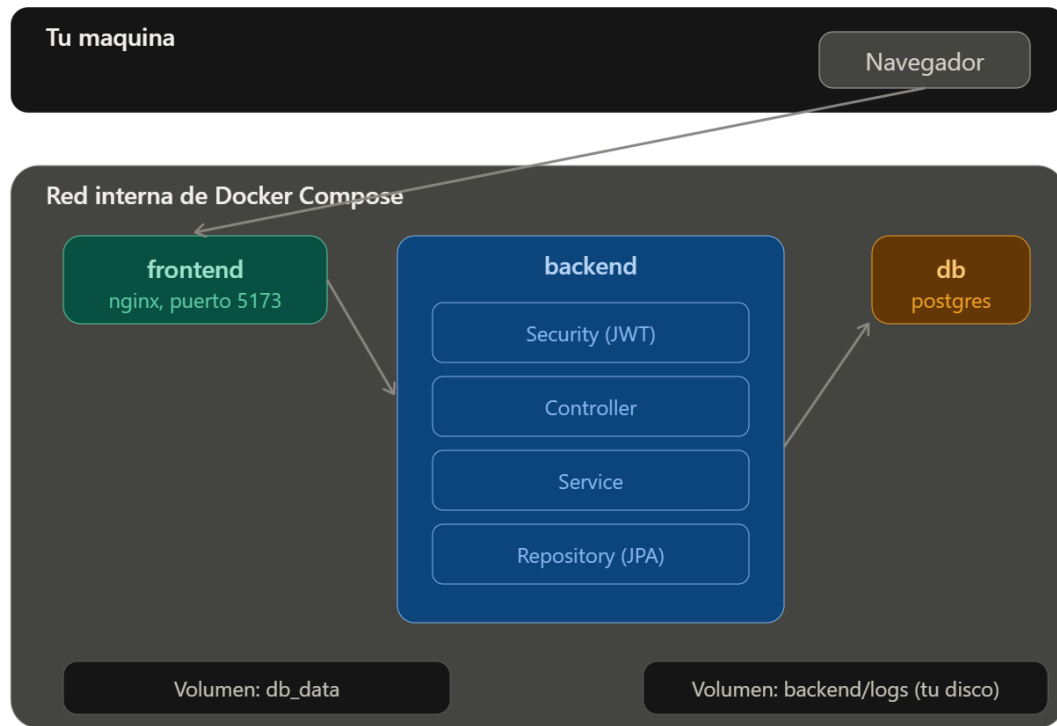
```
Prueba_Tecnica_Backend/
├── docker-compose.yml
├── backend/
│   ├── Dockerfile
│   ├── pom.xml
│   ├── openapi/openapi.yaml
│   └── src/main/
│       ├── java/com/telco/ventas/
│       │   ├── entity/ enums/ repository/
│       │   ├── dto/request/ dto/response/
│       │   ├── service/ controller/ security/
│       │   └── exception/ config/
│       └── resources/
│           ├── application.yml
│           ├── logback-spring.xml
│           └── db/ schema.sql, data.sql
├── frontend/ index.html, style.css, app.js
└── docs/     diagrama-solucion.md, decisiones-tecnicas.md
```

3. Arquitectura de la solución

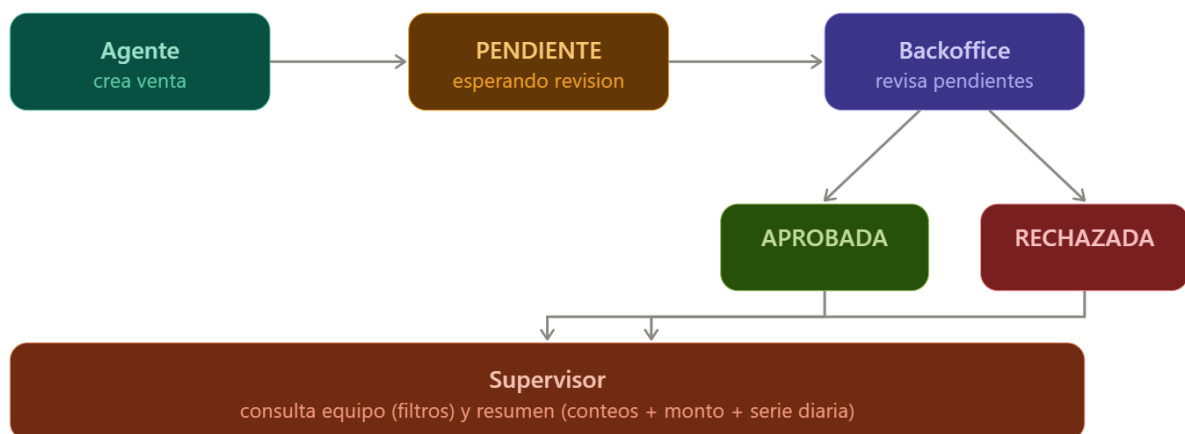
Tres contenedores Docker en red interna: frontend (nginx sirviendo HTML/CSS/JS plano), backend (Spring Boot con arquitectura en capas Controller → Service → Repository), y db (PostgreSQL). Los datos de la BD y los logs de auditoría persisten en volúmenes que sobreviven a un rebuild.



3.1 Arquitectura del sistema:

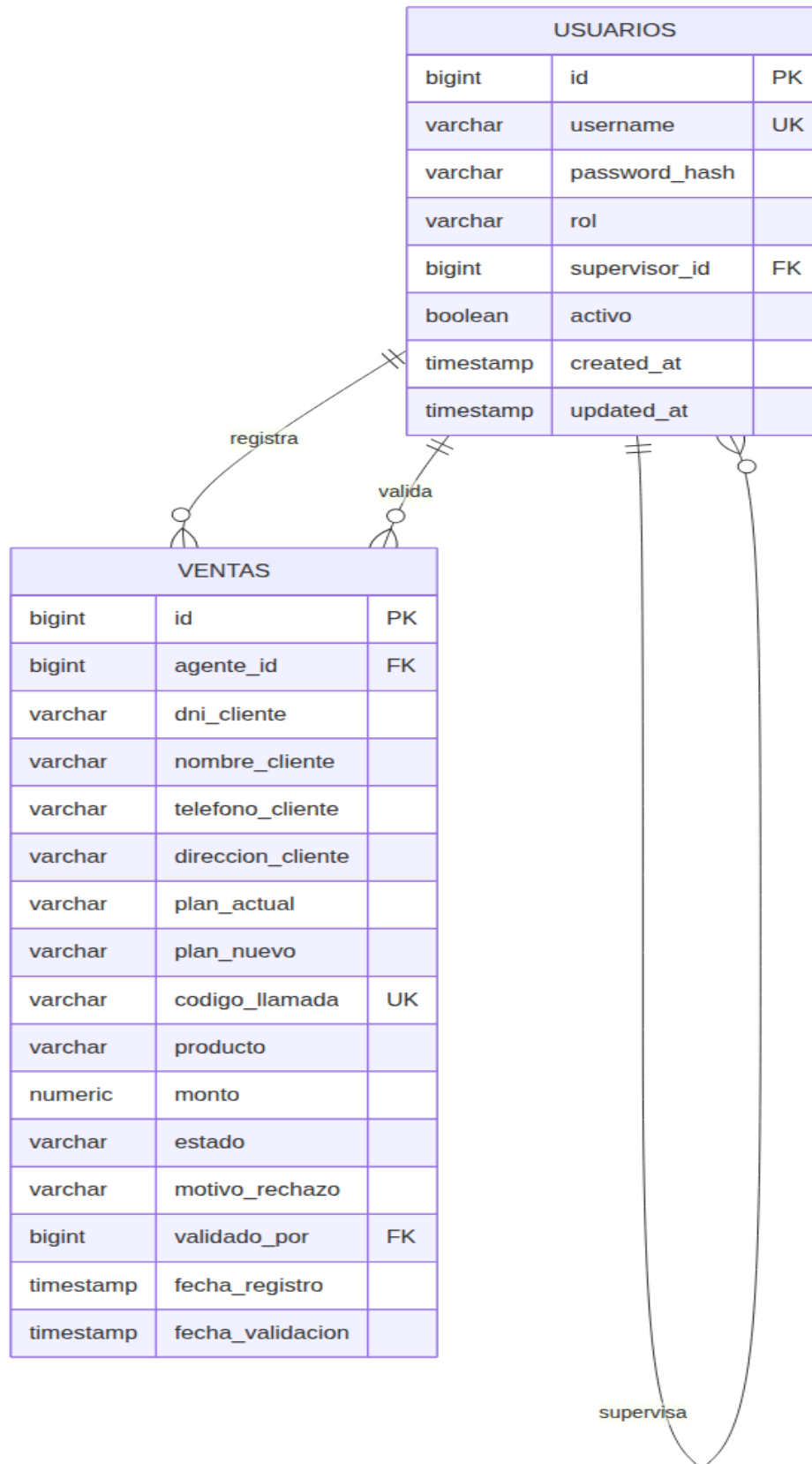


3.2 Flujo de procesos de ventas:



4. Modelo de datos

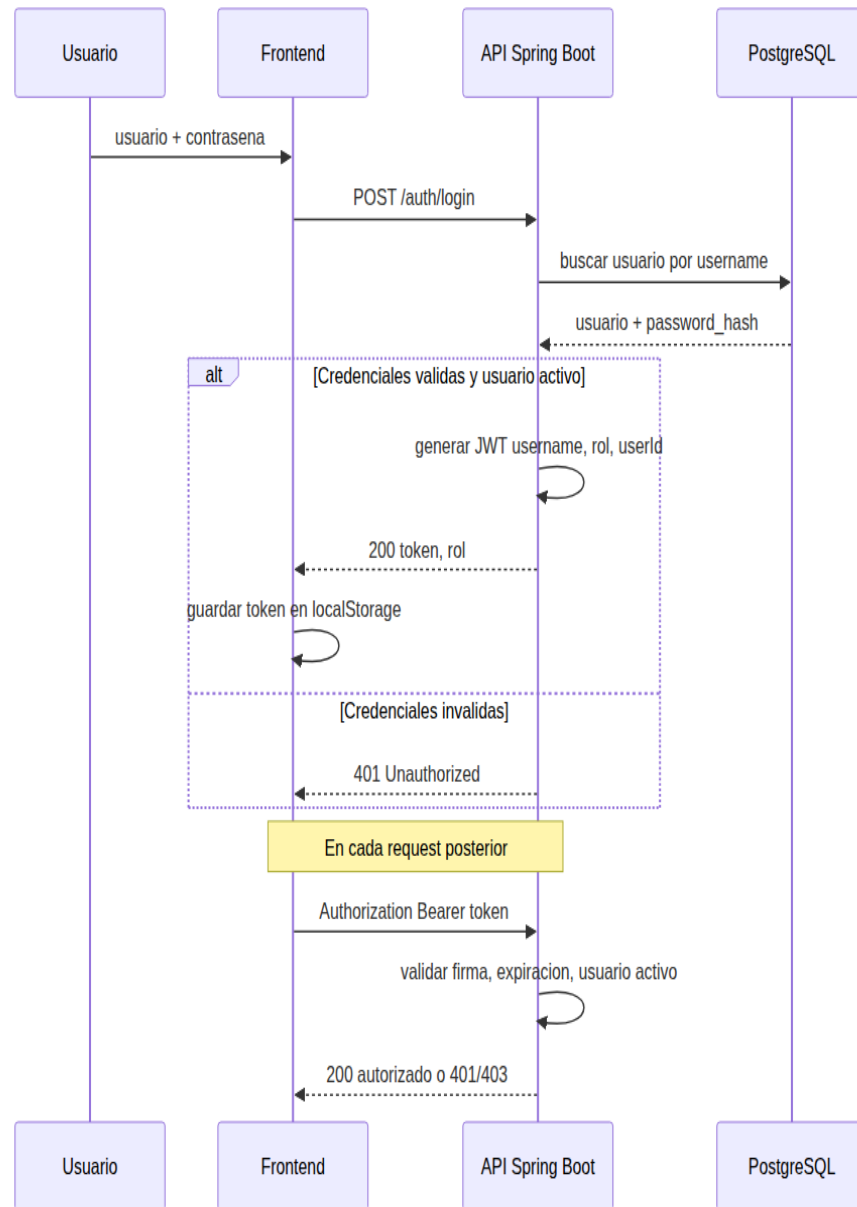
Dos tablas: usuarios (con auto-referencia a supervisor_id, modelando la jerarquia supervisor-agentes) y ventas (que guarda un snapshot de los datos del cliente al momento de la venta, en vez de referenciar una tabla clientes separada -decision documentada en la seccion 7).



5. Flujos de proceso

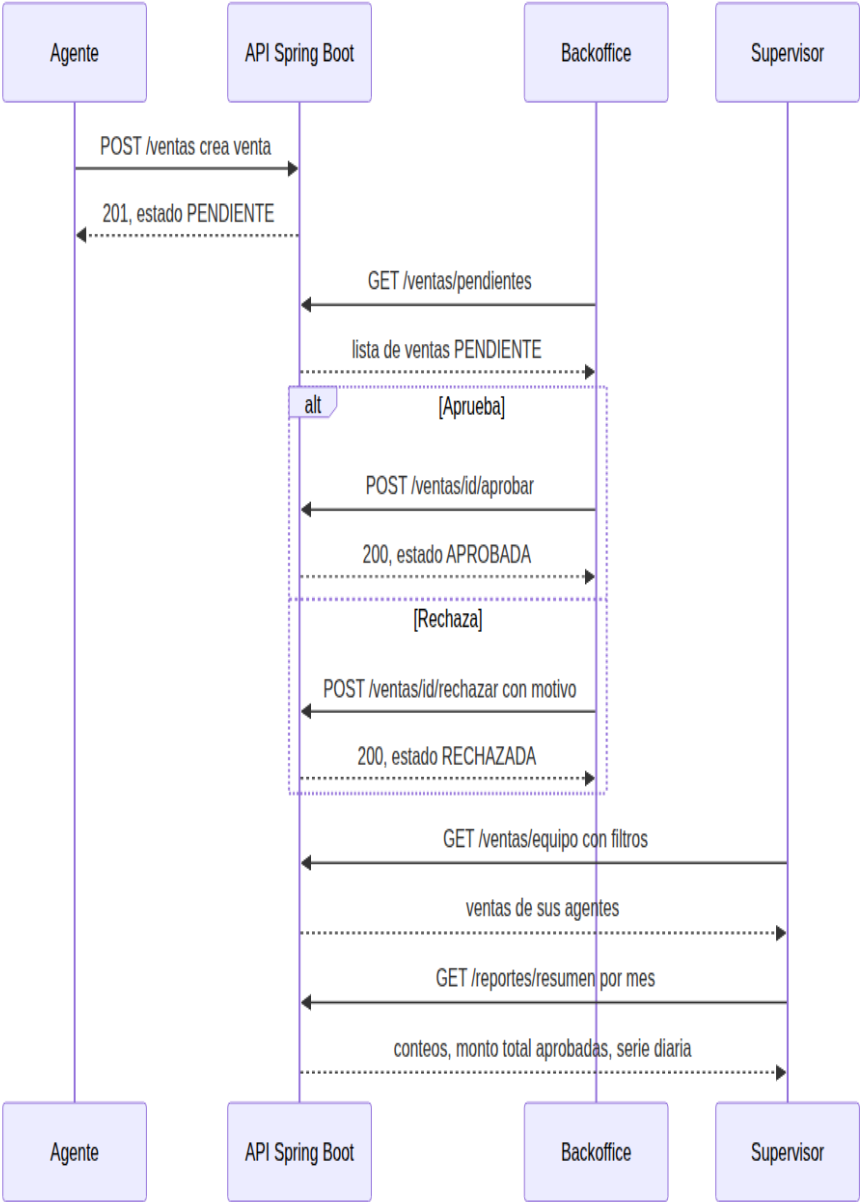
5.1 Autenticación (JWT)

Login stateless con JWT firmado HMAC. El token lleva username, rol y userId como claims. Cada request posterior se valida por firma, expiracion, y que el usuario siga activo -este ultimo chequeo evita que un usuario desactivado por el admin siga usando un token ya emitido.



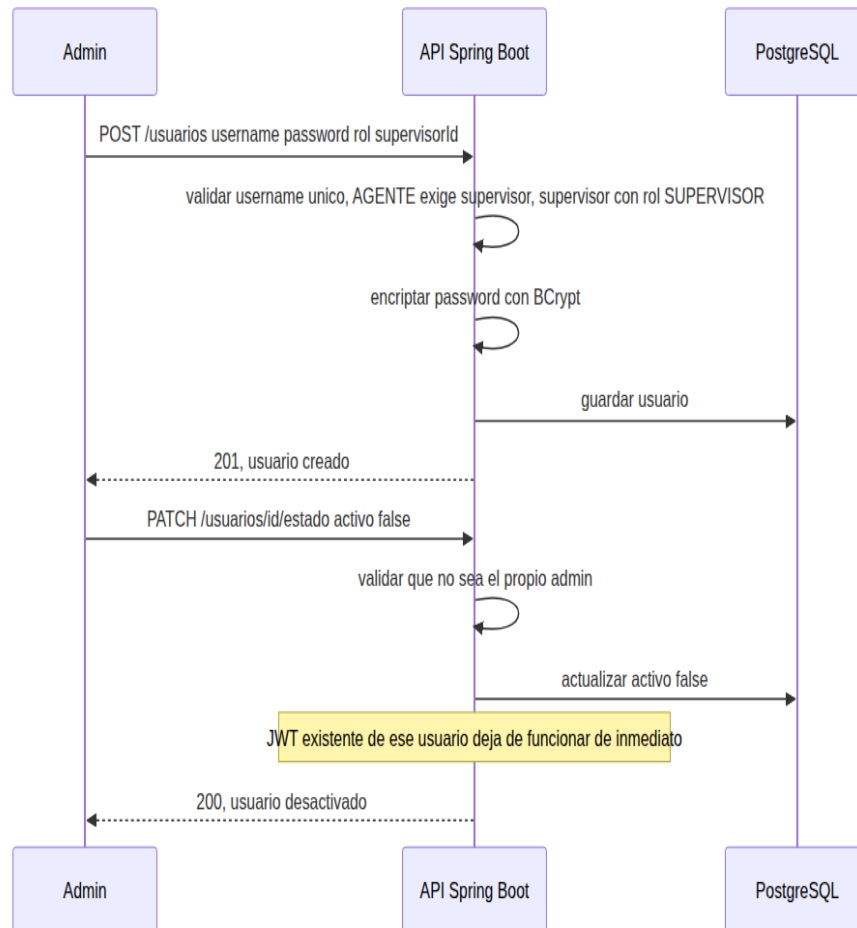
5.2 Proceso de venta

El circuito completo: registro por el agente, revision por backoffice (aprobar o rechazar con motivo obligatorio), y consulta por el supervisor (equipo filtrado + resumen agregado por periodo).



5.3 Administración de usuarios

El ADMIN crea usuarios con validaciones de negocio (username unico, un AGENTE siempre requiere un supervisor real con rol SUPERVISOR), y puede desactivarlos -con salvaguarda para que nunca se desactive a si mismo por accidente.



6. Endpoints principales

Prefijo /api/v1. Todos requieren header Authorization: Bearer <token> excepto /auth/login.

Método	Ruta	Rol	Descripción
POST	/auth/login	público	Devuelve JWT + rol
POST	/ventas	AGENTE	Crea venta (nace PENDIENTE)
GET	/ventas/mis-ventas	AGENTE	Filtros estado/desde/hasta, paginado
GET	/ventas/pendientes	BACKOFFICE	Ventas en estado PENDIENTE
POST	/ventas/{id}/aprobar	BACKOFFICE	Cambia a APROBADA
POST	/ventas/{id}/rechazar	BACKOFFICE	Requiere motivo_rechazo

GET	/ventas/equipo	SUPERVISOR	Filtros estado/agentId/desde/hasta
GET	/reportes/resumen	SUPERVISOR	Por rango o por mes
GET/POST	/usuarios	ADMIN	Listar / crear usuarios
PUT	/usuarios/{id}	ADMIN	Editar username/rol/supervisor
PATCH	/usuarios/{id}/estado	ADMIN	Activar / desactivar
PATCH	/usuarios/{id}/password	ADMIN	Resetear contraseña

7. Decisiones técnicas

7.1 Motor de base de datos: PostgreSQL

Se eligió sobre MySQL por soporte nativo y confiable de CHECK constraints (usados extensivamente para blindar reglas de negocio a nivel de esquema), integración directa con Docker Compose, y porque con Spring Data JPA el cambio de motor es transparente -no hay costo real de elegir uno u otro.

7.2 Modelo de datos: snapshot intencional en ventas

Las tablas cumplen 1FN/2FN/3FN de forma directa, con una excepción deliberada: ventas guarda un snapshot de los datos del cliente en vez de referenciar una tabla clientes normalizada. Esto es intencional, no un descuido -si el cliente cambia de dirección después, la venta histórica debe conservar los datos que tenía al momento de la venta, por auditoría. El enunciado además exige explícitamente de crear tabla de clientes.

7.3 Arquitectura del backend: capas MVC + Service

Controller (HTTP) -> Service (reglas de negocio) -> Repository (JPA) -> Entity, con DTOs de por medio que nunca exponen la entidad JPA directo. Los filtros dinámicos usan Spring Data Specifications. Las reglas de transición de estado (solo se aprueba/rechaza una venta PENDIENTE, rechazar exige motivo) viven en el Service, no en anotaciones de la entidad.

Nota técnica relevante: con open-in-view=false, la conversión de entidad a DTO debe ocurrir dentro del límite transaccional del Service -si se hace en el Controller (como se intentó inicialmente), las relaciones lazy (agente, validadoPor) fallan con LazyInitializationException al no haber sesión de Hibernate activa.

7.4 Seguridad: JWT stateless

Autenticación JWT (HMAC-SHA), sesiones stateless. El rol se traduce a authority ROLE_<ROL> para proteger endpoints declarativamente con @PreAuthorize. Dos gaps de seguridad se identificaron y corrigieron en revisión:

- El filtro JWT ahora verifica que el usuario siga activo en cada request, no solo la validez de la firma -de lo contrario, un usuario desactivado seguiría autenticado hasta que su token expire.
- El frontend escapa todo dato proveniente del backend antes de insertarlo en el DOM, previniendo XSS (ej. un nombre_cliente malicioso con HTML/JS embebido).

7.5 Manejo de errores

Un @RestControllerAdvice centraliza todas las excepciones y las traduce siempre al mismo formato JSON: { timestamp, path, error, message, details? }. Ninguna excepción no controlada llega al cliente como stack trace crudo; los errores inesperados sí quedan logueados completos en el servidor para poder diagnosticarlos.

7.6 Auditoría

Dos niveles: estructurada en BD (ventas.validado_por + fecha_validacion) y de eventos en logs, con loggers separados por dominio (AUDIT.VENTAS, AUDIT.AUTH, AUDIT.USUARIOS) que escriben en archivos independientes bajo logs/audit/, persistidos en un volumen Docker para sobrevivir a rebuilds.

8. Guía de despliegue local

8.1 Todo en Docker (recomendado)

Desde la raíz del repositorio:

```
docker compose up --build
```

Levanta los 3 servicios (db, backend, frontend). Sin el flag -d, la terminal queda pegada mostrando los logs en vivo, igual que antes con `mvn spring-boot:run`. El backend imprime un banner con las URLs clickeables al terminar de arrancar.

Abrir directo: <http://localhost:5173>

8.2 Usuarios de prueba (seed)

Username	Password	Rol	Notas
admin	Admin*123	ADMIN	
supervisor1	Sup*123	SUPERVISOR	Supervisa a agente1 y agente2
agente1	Agente*123	AGENTE	Bajo supervisor1
agente2	Agente*123	AGENTE	Bajo supervisor1
back1	Back*123	BACKOFFICE	

8.3 Logs

Dos formas de verlos: archivos en disco (`backend/logs/`, gracias al volumen Docker) para revisar auditoria directo con cualquier editor, o `docker compose logs -f backend` para la consola en vivo del contenedor.

9. Entregables cumplidos

- Repositorio con `backend/` y `frontend/`
- `README.md` con pasos de ejecucion Windows/Linux, BD incluida
- `schema.sql` + `data.sql` con los 4 usuarios seed y ventas variadas
- OpenAPI completo (`backend/openapi/openapi.yaml`) validado sintacticamente
- `docs/` con diagramas (este documento) y decisiones tecnicas